Name: Eria Mutasa

Course: CYB615 — Penetration Testing

Instructor: Prof. M Darling

Due Date: March 26, 2026

Lab 3: Network Traffic Analysis with Wireshark

HTTP Plaintext vs. HTTPS Encrypted Traffic Capture — March 26, 2026

1. Overview & Objectives

Wireshark is the industry-standard tool for capturing and analyzing network traffic. This lab exercise demonstrates one of the most critical concepts in network security: the exposure of credentials transmitted over unencrypted HTTP versus the confidentiality protection provided by HTTPS/TLS. By capturing a live login over HTTP against DVWA (Damn Vulnerable Web Application) and comparing it with encrypted HTTPS traffic from an external site, the lab provides hands-on evidence of why TLS is mandatory for any web application that handles authentication or sensitive data.

The objectives of this exercise were to:

1. Capture a live HTTP POST request containing plaintext credentials using Wireshark on the loopback interface.
2. Compare unencrypted HTTP traffic with TLS-encrypted HTTPS traffic and document the difference in readability.
3. Analyze the security implications of unencrypted credential transmission from an attacker and penetration tester perspective.

NOTE	All testing was conducted on a Kali Linux VM in a controlled lab environment. DVWA (Damn Vulnerable Web Application) was used as the HTTP target. No real external systems were attacked or compromised.

2. Captured Traffic Evidence

2.1 Screenshot 1 — HTTP Plaintext Credentials

The capture below shows a live HTTP POST request intercepted by Wireshark on the loopback (lo) interface. With the filter `http.request.method == POST` applied, the URL Encoded Form Data section of the selected packet reveals the submitted username and password in fully readable plaintext. A date terminal is visible in the background to timestamp the capture session.

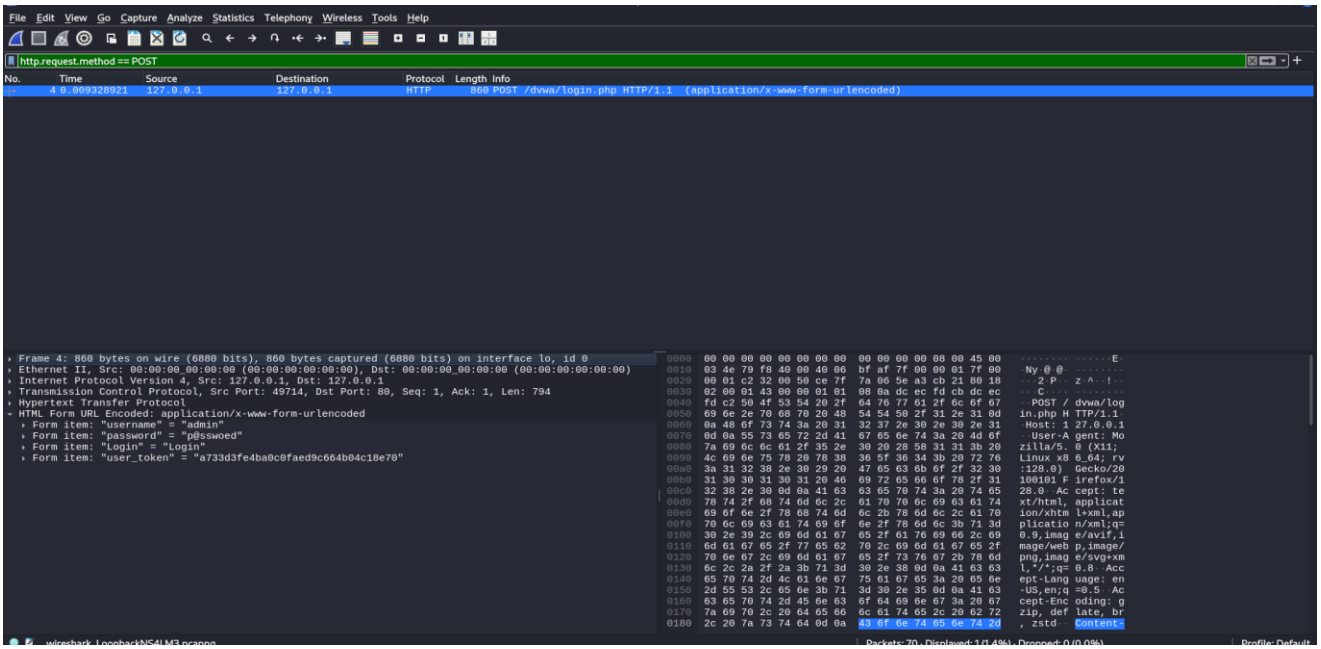

Figure 1 — Wireshark capture on the loopback interface showing an HTTP POST request to DVWA login.php. The URL Encoded Form Data section reveals username and password in plaintext.

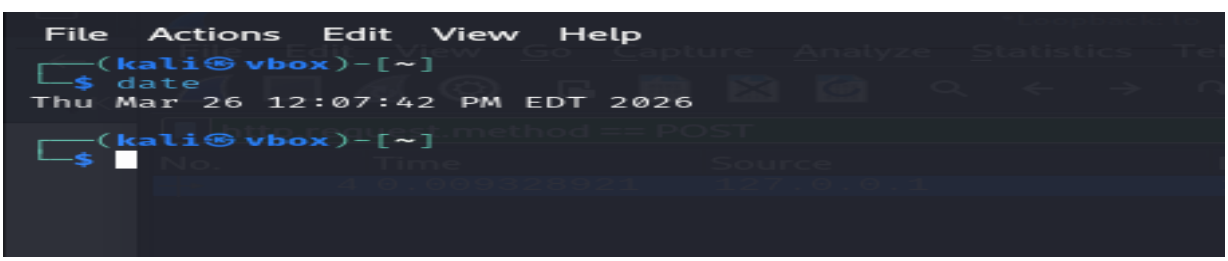

Figure 2 — Date terminal output providing a system timestamp to corroborate the capture session shown in Figure 1.

RESULT	Interface: lo (loopback) | Filter: http.request.method == POST | Credentials visible in URL Encoded Form Data

2.2 Screenshot 2 — HTTPS / TLS Encrypted Traffic

After restarting Wireshark on the external interface and browsing to `https://www.google.com`, the capture below shows the resulting TLS traffic. Applying the `tls` filter reveals encrypted ciphertext; no URLs, credentials, or readable content are exposed. An attempt to filter for `http.request.method == POST` returns zero results, confirming all POST data is wrapped inside the TLS layer.

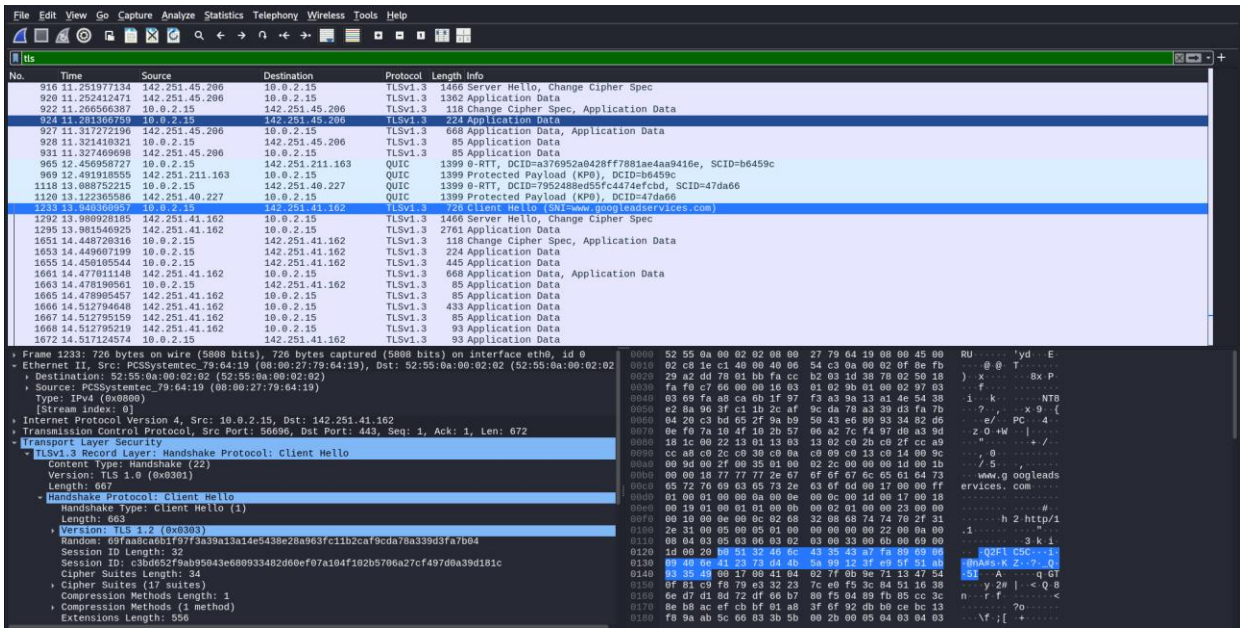

Figure 3 — Wireshark capture on the external interface with the tls filter applied. Packet contents are unreadable ciphertext; no credentials or request data are visible.

RESULT	Interface: eth0/enp0s1 (external) | Filter: tls | All packet contents encrypted; http.request.method == POST returns zero results

3. Analysis Questions

3.1 HTTP Security Concerns

What did you see in the HTTP capture that concerns you? Why is it a problem? In the HTTP capture, I observed that both the username and password were transmitted in plaintext and were clearly visible in the URL Encoded Form Data section of the POST request. This is a critical security vulnerability because anyone with network access can intercept and read these credentials without any decryption effort. If an attacker captures this traffic, they gain immediate access to the user's account, potentially leading to unauthorized access, data theft, or further system compromise. Unlike encrypted traffic, HTTP provides no confidentiality protection, making credential theft trivially easy.

3.2 Wi-Fi Traffic Interception

If you were on the same Wi-Fi as someone at a coffee shop, could you see their traffic with Wireshark? Under what conditions? Yes, it is possible to capture someone else's traffic on a shared Wi-Fi network, but it depends on several conditions. On older or misconfigured networks using WEP encryption or open Wi-Fi, capturing traffic is straightforward. On modern WPA2/WPA3 networks, you would need to use techniques like ARP spoofing or setting your wireless card to promiscuous/monitor mode to intercept packets not addressed to your device. However, even if you capture the packets, HTTPS-encrypted traffic would still be unreadable without the encryption keys. This demonstrates why using HTTPS and VPNs on public Wi-Fi is essential, even if packets are intercepted, the content remains protected.

3.3 Wireshark in Penetration Testing

As a penetration tester, how would you use Wireshark during an engagement? Give one specific example. As a penetration tester, I would use Wireshark to identify unencrypted protocols and credentials during network reconnaissance and exploitation phases. For example, after gaining access to a network segment, I would capture traffic to look for legacy protocols like FTP, Telnet, or HTTP that transmit credentials in plaintext. If I identify a device or application using HTTP for authentication, I could demonstrate the risk by capturing real credentials and documenting them in the penetration test report. This provides concrete evidence of vulnerabilities and helps justify security improvements like requiring HTTPS or implementing network segmentation to protect sensitive traffic.

4. Conclusion

This lab successfully demonstrated the fundamental difference between unencrypted HTTP and encrypted HTTPS traffic. The ability to easily capture plaintext credentials in HTTP traffic highlights why encryption is critical for protecting sensitive data, especially on shared or public networks. Modern security practices mandate HTTPS for all web applications handling authentication or sensitive information, and tools like Wireshark help security professionals identify and remediate these vulnerabilities during penetration testing engagements.

KEY TAKEAWAY	HTTP transmits credentials and request data in plaintext readable by any observer on the network. HTTPS wraps all traffic in TLS, making interception useless without the session keys. Any web application handling authentication must use HTTPS without exception.